ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ВНУТРІШНІХ СПРАВ

Кафедра інформаційних систем та технологій

ЛАБОРАТОРНА РОБОТА №5

з дисципліни «Об’єктно-орієнтовне програмування»

Тема: «Поліморфізм та перевизначення методів»

Виконав:

курсант 2-го курсу

ННІ 4-25-203 інф

Кожевніков К.О.

Прийняв:

Доцент кафедри ІСТ, к.т.н.,
Слободянюк О.В.

КАМ’ЯНЕЦЬ-ПОДІЛЬСЬКИЙ – 2026

ВАРІАНТ 6. Медичний центр

Завдання:

Класи для роботи: Service, Appointment, Client. Створіть Consultation та DiagnosticVisit зі спільним методом preparation_note(). Використовуйте вигадані дані й не додавайте персональні відомості.

КОД РОЗВ'ЯЗКУ

```
class Service:
    """Модель медичної послуги."""

    def __init__(self, name, duration_min):
        self.name = name
        self.duration_min = duration_min

    def show_service(self):
        """Повертає інформацію про послугу."""
        return f"{self.name} ({self.duration_min} хв)"


class Client:
    """Модель пацієнта (клієнта) медичного центру."""

    def __init__(self, full_name, phone):
        self.full_name = full_name
        self.phone = phone

    def show_client(self):
        """Повертає інформацію про клієнта."""
        return f"{self.full_name} (тел: {self.phone})"


class Appointment:
    """Базова модель запису на прийом із контрольованим доступом до
статусу."""

    valid_statuses = {"підтверджено", "завершено", "скасовано"}
    created_count = 0  # Атрибут класу для підрахунку створених записів

    def __init__(self, number, time, service, client, status="підтверджено"):
        self.number = number
        self.time = time
        self.service = service  # Екземпляр класу Service
        self.client = client    # Екземпляр класу Client

        self._status = "підтверджено"  # Початкове значення перед викликом
setter
```

```
        self.status = status           # Запис проходить через setter із
перевіркою

        Appointment.created_count += 1

    @property
    def status(self):
        """Повертає поточний статус із захищеного внутрішнього поля."""
        return self._status

    @status.setter
    def status(self, value):
        """Дозволяє встановити лише допустимий статус."""
        if value not in self.valid_statuses:
            raise ValueError(f"Невідомий статус: {value}")
        self._status = value

    def show(self):
        """Повертає загальну інформацію про запис."""
        return (
            f"Запис №{self.number} | Час: {self.time} | "
            f"Клієнт: {self.client.show_client()} | "
            f"Послуга: {self.service.show_service()} | Статус: {self.status}"
        )

    def preparation_note(self):
        """Повертає загальну інструкцію з підготовки до прийому."""
        return "З'явитися за 10 хвилин до визначеного часу з документами."


class Consultation(Appointment):
    """Похідний клас для консультаційних прийомів лікаря."""

    def __init__(self, number, time, service, client, doctor_office,
status="підтверджено"):
        super().__init__(number, time, service, client, status)
        self.doctor_office = doctor_office  # Номер кабінету лікаря

    def preparation_note(self):
        """Перевизначений поліморфний метод для консультації."""
        # Використовуємо базову інструкцію та адаптуємо її для консультації
        base_note = super().preparation_note()
        return f"Підготовка до консультації (каб. №{self.doctor_office}):
очікуйте під кабінетом. {base_note}"


class DiagnosticVisit(Appointment):
    """Похідний клас для діагностичних процедур (УЗД, аналізи тощо)."""
```

```
    def __init__(self, number, time, service, client, equipment_code,
status="підтверджено"):
        super().__init__(number, time, service, client, status)
        self.equipment_code = equipment_code  # Інвентарний код обладнання

    def preparation_note(self):
        """Перевизначений поліморфний метод для діагностики."""
        return (
            f"Підготовка до діагностики (обладнання код:
{self.equipment_code}): "
            f"дотримуйтеся спеціальних рекомендацій лікаря перед процедурою."
        )


# Демонстрація поліморфізму
if __name__ == "__main__":
    # Створюємо об'єкти послуг
    s1 = Service("Консультація терапевта", 30)
    s2 = Service("УЗД черевної порожнини", 40)
    s3 = Service("Консультація кардіолога", 30)

    # Створюємо вигадані об'єкти клієнтів
    c1 = Client("Олександр Т.", "+380501112233")
    c2 = Client("Марія Д.", "+380672223344")
    c3 = Client("Іван К.", "+380933334455")

    # Створюємо список різнотипних екземплярів
    appointments = [
        Consultation(1, "2026-11-02 09:00", s1, c1, "204", "підтверджено"),
        DiagnosticVisit(2, "2026-11-02 10:30", s2, c2, "EQ-9988",
"підтверджено"),
        Consultation(3, "2026-11-02 12:00", s3, c3, "305", "підтверджено")
    ]

    print("=== Демонстрація поліморфізму: виклик preparation_note() у циклі
===")
    # Викликаємо метод для всіх об’єктів одним циклом без розгалужень за
класом
    for app in appointments:
        print(f"Запис №{app.number} ({app.service.name}):")
        print(f" -> {app.preparation_note()}")
        print("-" * 60)
```

РЕЗУЛЬТАТИ РОБОТИ ПРОГРАМИ

```
PS C:\Users\user\AppData\Local\Programs\Microsoft VS Code> & C:\Users\user\AppData\Local\Programs\Python\Python38-32\python.exe c:/Users/user/D
=== Демонстрація поліморфізму: виклик preparation_note() у циклі ===
Запис №1 (Консультація терапевта):
 -> Підготовка до консультації (каб. №204): очікуйте під кабінетом. З'явитися за 10 хвилин до визначеного часу з документами.
------------------------------------------------------------
Запис №2 (УЗД черевної порожнини):
 -> Підготовка до діагностики (обладнання код: EQ-9988): дотримуйтеся спеціальних рекомендацій лікаря перед процедурою.
------------------------------------------------------------
Запис №3 (Консультація кардіолога):
 -> Підготовка до консультації (каб. №305): очікуйте під кабінетом. З'явитися за 10 хвилин до визначеного часу з документами.
```